HARI 2 • MATERI 1

Form input dan validasi data di sisi server

Membangun formulir dengan komponen Bootstrap 5, membaca kiriman data POST di Express, serta memeriksa keamanan input menggunakan `express-validator`.

Kode: TX-BE01 **45 menit Materi 1 & Lab 1**

POKOK BAHASAN SESI INI

01 Menyusun antarmuka `<form>` dengan kelas Bootstrap 5

02 Membaca kiriman formulir lewat `express.urlencoded`

03 Perbandingan validasi sisi peramban dan sisi server

04 Aturan pemeriksaan dengan pustaka `express-validator`

05 Menampilkan pesan kesalahan & praktik Lab 1

PETA BELAJAR

Lima langkah mengolah input pengguna secara aman

Membuka Hari 2 dengan alur penerimaan data formulir dari peramban hingga pengecekan di server Express.

01 Form Bootstrap 5

Menyusun kolom input yang rapi dan seragam menggunakan kelas `form-control`, `mb-3`, dan `btn`.

02 Middleware pembaca form

Mengaktifkan `express.urlencoded({ extended: true })` agar isi formulir terbaca di dalam `req.body`.

03 Pertahanan sisi server

Memahami prinsip *Never trust user input* serta perbedaan validasi *frontend* dan *server*.

04 Pustaka `express-validator`

Menyusun aturan pengecekan (`notEmpty()`, `isEmail()`) dan memeriksa hasil dengan `validationResult`.

05 Pesan kesalahan & Lab 1

Mengirim daftar pesan kesalahan kembali ke halaman `views/form.ejs` menggunakan kotak peringatan Bootstrap.

01 • FORM BOOTSTRAP

Tiga atribut utama pada elemen <form> dan <input>

Untuk mengumpulkan data dari pengguna (seperti registrasi atau tambah barang), kita menyusun elemen formulir dengan tiga pengatur utama:

01 TUJUAN KIRIM

action="/submit-form"

Menentukan alamat rute di server Express yang akan menerima dan memproses data saat tombol kirim ditekan.

> Contoh: diarahkan ke rute `app.post('/submit-form', ...)`.

02 METODE HTTP

method="POST"

Mengirimkan data formulir di dalam tubuh permintaan (*request body*) agar isinya tidak terpampang di alamat URL.

> Contoh: aman untuk mengirim nama, email, atau kata sandi.

03 IDENTITAS KOLOM

name="nama"

Menjadi nama properti saat data dibaca di sisi server. Tanpa atribut `name`, nilai kolom tidak akan ikut terkirim.

> Contoh: `name="email"` dibaca lewat `req.body.email`.

01 • KELAS TAMPILAN FORMULIR

Merapikan komponen formulir dengan kelas Bootstrap 5

Bootstrap menyediakan kelas siap pakai agar kotak input, label, dan pemberitahuan tampil konsisten di berbagai ukuran perangkat:

KELAS BOOTSTRAP	ELEMEN SASARAN	FUNGSI PADA TAMPILAN FORMULIR
`form-control`	`<input>`, `<select>`, `<textarea>`	Memberikan lebar penuh (100%), sudut membulat, garis tepi rapi, dan efek sorot biru saat diklik.
`mb-3`	`<div class="mb-3">`	Memberi jarak bawah (*margin-bottom*) antarkelompok label dan kolom input agar tidak saling menempel.
`alert alert-danger`	`<div class="alert ...">`	Menampilkan kotak pemberitahuan berwarna merah muda saat terdapat kesalahan pengisian data.
`btn btn-primary`	`<button type="submit">`	Mengubah tombol kirim standar menjadi tombol biru khas Bootstrap yang jelas dan mudah dikenali.

> **Kenyamanan pengguna:** Tata letak yang bersih dan pesan peringatan yang jelas membantu pengguna memperbaiki kesalahan input dengan cepat.

02 • KONFIGURASI PEMBACA FORM

Mengaktifkan `express.urlencoded` pada `app.js`

Secara bawaan, Express belum mengurai isi tubuh permintaan dari formulir HTML. Kita perlu menambahkan satu baris *middleware* terlebih dahulu:

TANPA MIDDLEWARE PEMBACA FORM

`req.body` bernilai `undefined`

Saat pengguna menekan tombol Kirim, server menerima permintaan `POST` tetapi tidak dapat membaca nilai `req.body.nama` maupun `req.body.email`.

DENGAN EXPRESS.URLENCODED({ EXTENDED: TRUE })

Data formulir siap diakses lewat `req.body`

Express otomatis mengubah data kiriman formulir menjadi objek JavaScript yang rapi, misalnya `{ nama: 'Budi', email: 'budi@mail.com' }`.

Pastikan baris `app.use(express.urlencoded({ extended: true }));` diletakkan di bagian atas `app.js` sebelum daftar rute ditulis.

03 • KEAMANAN INPUT PENGGUNA

Prinsip utama: jangan pernah mempercayai input pengguna

Data yang dikirim dari luar (*user input*) bisa saja kosong, salah format, atau disisipi kode berbahaya. Karena itu kita mengenal dua lapis validasi:

LAPIS PERTAMA • SISI PERAMBAN

Validasi Frontend

Pemeriksaan awal langsung di peramban sebelum formulir dikirimkan ke server.

- Menggunakan atribut HTML bawaan seperti `required` atau `type="email"`.
- Memberi umpan balik cepat kepada pengguna tanpa perlu menunggu balasan server.
- **Kelemahan:** Mudah dilewati jika pengguna mematikan fitur di peramban atau mengirim permintaan langsung.

Atribut required | **Cepat di Browser** | **Bisa Dilewati**

LAPIS PERTAHANAN UTAMA • NODE.JS

Validasi Server

Pemeriksaan wajib di sisi server Node.js setelah data dikirim dan diterima oleh rute Express.

- Tidak dapat diakali dari sisi peramban karena berjalan sepenuhnya di mesin server.
- Memastikan kolom tidak kosong, format email benar, dan data sesuai tipe sebelum disimpan.
- Menggunakan pustaka populer `express-validator` pada rute `POST`.

express-validator | **Pertahanan Utama** | **Aman & Terkendali**

04 • PUSTAKA EXPRESS-VALIDATOR

Dua fungsi utama dari pustaka express-validator

Setelah menjalankan perintah `npm install express-validator`, kita mengimpor dua fungsi utama ke dalam berkas `app.js`:

INSTALASI & KONFIGURASI DI `APP.JS`

```
// 1. Perintah di terminal:
// npm install express-validator

// 2. Tambahkan di app.js:
app.use(express.urlencoded({ extended: true }));

const {
  body,
  validationResult
} = require('express-validator');
```

Kedua fungsi diambil langsung dari paket `express-validator`.

01 **Fungsi `body('namaField')`**

Menentukan kolom input mana di dalam `req.body` yang hendak diperiksa beserta rantai aturan syaratnya.

02 **Metode `.withMessage('...')`**

Menentukan kalimat pesan kesalahan yang jelas jika syarat validasi tepat di depannya gagal dipenuhi.

03 **Fungsi `validationResult(req)`**

Mengumpulkan hasil pemeriksaan dari objek `req` untuk dicek apakah terdapat pelanggaran aturan.

04 • RANTAI ATURAN VALIDASI

Menuliskan aturan pemeriksaan pada rute POST

Aturan validasi disisipkan sebagai parameter kedua berbentuk array di antara nama rute dan fungsi penangan `(req, res)`:

RUTE POST DI `APP.JS`

```
app.post('/submit-form', [
  body('nama')
    .notEmpty()
    .withMessage('Nama tidak boleh kosong'),
  body('email')
    .isEmail()
    .withMessage('Format email tidak valid')
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.render('form', {
      errors: errors.array()
    });
  }
  res.send(`Sukses! Nama: ${req.body.nama}, Email:
${req.body.email}`);
});
```

01 `body('nama').notEmpty()`

Memastikan kolom bernama `nama` tidak dibiarkan kosong saat pengguna mengirim formulir.

02 `body('email').isEmail()`

Memeriksa apakah isian kolom `email` sesuai dengan format alamat email yang benar.

03 `if (!errors.isEmpty())`

Jika ada kesalahan, hentikan proses dengan `return` lalu kirim kembali halaman `form` bersama daftar `errors.array()`.

05 • MENAMPILKAN PESAN ERROR DAN NOTIFIKASI

Merender kotak peringatan kesalahan di `views/form.ejs`

Saat input ditolak, aplikasi mengirim pengguna kembali ke halaman form dan menampilkan daftar letak kesalahan di bagian atas halaman:

01 **Cek `typeof errors != 'undefined'`**

Saat `GET /form` pertama kali dibuka, variabel `errors` belum dikirim. Pengecekan ini menjaga halaman tetap aman.

02 **Kotak `alert alert-danger`**

Membungkus daftar pesan kesalahan di dalam komponen kotak peringatan merah Bootstrap agar langsung terlihat jelas.

03 **Cetak properti `<%= err.msg %>`**

Setiap item di dalam `errors` menyimpan teks pesan dari `.withMessage()` pada properti `msg`.

BLOK PERINGATAN DI `VIEWS/FORM.EJS`

```
<!-- Area Menampilkan Error -->
<% if(typeof errors != 'undefined') { %>
  <div class="alert alert-danger">
    <ul>
      <% errors.forEach(function(err) { %>
        <li><%= err.msg %></li>
      <% }) %>
    </ul>
  </div>
<% } %>
```

Jika tidak ada kesalahan, blok `alert-danger` ini tidak ditampilkan di peramban.

SIMULASI INTERAKTIF • UJI SKENARIO INPUT

Simulasi hasil pemeriksaan express-validator

1. Dikirim Kosong | 2. Format Email Salah | 3. Semua Input Valid

SKENARIO 1 • DITOLAK SERVER

Pengguna menekan tombol Kirim tanpa mengisi kolom apa pun

Kedua aturan gagal dipenuhi: kolom nama kosong dan kolom email bukan format email yang sah. Server mengembalikan halaman form.ejs beserta 2 pesan kesalahan.

ISI ERRORS.ARRAY() & RESPON SERVER

```
// Data diterima: { nama: '', email: '' }
// Hasil errors.array():
[
  { path: 'nama', msg: 'Nama tidak boleh kosong' },
  { path: 'email', msg: 'Format email tidak valid' }
]

// Server merender ulang halaman form:
return res.render('form', { errors: errors.array() });
```

Pengguna melihat kotak merah berisi 2 poin kesalahan di atas formulir.

SIMULASI INTERAKTIF • UJI SKENARIO INPUT

Simulasi hasil pemeriksaan express-validator

1. Dikirim Kosong | **2. Format Email Salah** | 3. Semua Input Valid

SKENARIO 2 • DITOLAK SERVER

Nama sudah diisi, tetapi email diketik tanpa tanda @

Aturan body('nama').notEmpty() berhasil lolos, namun body('email').isEmail() gagal. Server tetap menahan proses dan menampilkan 1 pesan kesalahan.

ISI ERRORS.ARRAY() & RESPON SERVER

```
// Data diterima: { nama: 'Budi Santoso', email:
'budi.santoso' }
// Hasil errors.array():
[
  { path: 'email', msg: 'Format email tidak valid' }
]

// Server merender ulang halaman form:
return res.render('form', { errors: errors.array() });
```

Hanya aturan yang dilanggar yang masuk ke dalam daftar `errors.array()`.

SIMULASI INTERAKTIF • UJI SKENARIO INPUT

Simulasi hasil pemeriksaan express-validator

1. Dikirim Kosong | 2. Format Email Salah | **3. Semua Input Valid**

SKENARIO 3 • LOLOS VALIDASI

Nama terisi dan format email ditulis dengan benar

Seluruh aturan terpenuhi sehingga errors.isEmpty() bernilai true. Blok if dilewati dan server memproses data formulir dari req.body dengan aman.

RESPON SUKSES DARI SERVER

```
// Data diterima: { nama: 'Budi Santoso', email:
'budi@inixindo.id' }
// Hasil errors.isEmpty() -> true

res.send(
  `Sukses! Nama: ${req.body.nama}, Email:
${req.body.email}`
);
// Tampil: Sukses! Nama: Budi Santoso, Email:
budi@inixindo.id
```

Pada materi berikutnya, di titik inilah data yang bersih akan disimpan ke dalam *database*.

PERSIAPAN LAB 1 • BEDAH KODE VIEW

Struktur lengkap berkas `views/form.ejs`

Menggabungkan area pemberitahuan kesalahan dengan formulir input Bootstrap 5:

BERKAS `VIEWS/FORM.EJS`

```
<body class="container mt-5">
  <h2>Form Tambah Data</h2>
  <% if(typeof errors != 'undefined') { %>
    <div class="alert alert-danger">
      <ul>
      <% errors.forEach(function(err) { %>
        <li><%= err.msg %></li>
      <% }) %>
      </ul>
    </div>
  <% } %>
  <form action="/submit-form" method="POST">
    <div class="mb-3">
      <label>Nama:</label>
      <input type="text" name="nama" class="form-control">
    </div>
    <div class="mb-3">
      <label>Email:</label>
      <input type="text" name="email" class="form-control">
    </div>
    <button type="submit" class="btn btn-
primary">Kirim</button>
  </form>
</body>
```

01 **Satu tampilan untuk dua rute**

Berkas `form.ejs` dipakai saat membuka `GET /form` maupun saat mengembalikan pesan kesalahan dari `POST /submit-form`.

02 **Kesesuaian atribut** `name`

Nilai `name="nama"` dan `name="email"` harus sama persis dengan nama yang diperiksa pada `body('nama')` dan `body('email')`.

03 **Tombol** `type="submit"`

Memicu pengiriman seluruh isian formulir ke rute `/submit-form` dengan metode `POST`.

PRAKTIK MANDIRI • LAB 1 HARI 2

Saatnya praktik: membuat dan menguji validasi form

Ikuti langkah **Lab 1: Validasi Form** pada proyek Express Anda:

1. Pasang pustaka validator di terminal: `npm install express-validator`.
2. Tambahkan `app.use(express.urlencoded({ extended: true }))` serta impor `{ body, validationResult }` di `app.js`.
3. Buat berkas `views/form.ejs` berisi blok peringatan `errors` dan formulir input Nama serta Email.
4. Tambahkan rute `app.get('/form', ...)` dan `app.post('/submit-form', ...)` di dalam `app.js`.
5. *Restart* server, lalu buka dan uji alamat berikut di peramban:

```
http://localhost:3000/form
```

Cobalah menekan tombol **Kirim** tanpa mengisi apa-apa untuk melihat pesan kesalahan yang muncul, lalu uji kembali dengan data yang benar.

TARGET LAB 1

Validasi menolak input yang salah

Saat formulir dikirim kosong, tampil kotak merah berisi daftar kesalahan. Saat diisi dengan benar, tampil pesan `Sukses! Nama: ..., Email: ...`.

SESI BERIKUTNYA

Setelah mampu menyaring input pengguna, pada **Materi 2 Hari 2** kita akan menyiapkan *database* **PostgreSQL Cloud** di layanan Render.

HARI 2 • MATERI 2

PostgreSQL Cloud di Render dan keamanan berkas .env

Menyiapkan *database* relasional di *cloud* tanpa instalasi lokal, memahami struktur *Connection String*, serta mengamankan kredensial rahasia menggunakan `dotenv`.

Kode: TX-BE01 **35 menit Materi 2 & Lab 2**

POKOK BAHASAN SESI INI

01 Mengapa aplikasi membutuhkan *database* permanen

02 PostgreSQL & konsep *Database as a Service* (Render)

03 Membedah anatomi *Connection String* (`postgresql://...`)

04 Mengamankan kredensial di dalam berkas `.env`

05 Membaca konfigurasi dengan `dotenv` & praktik Lab 2

02

PETA BELAJAR

Lima langkah menyiapkan *database cloud* dan konfigurasi aman

Beralih dari penyimpanan array sementara di memori menuju penyimpanan permanen di PostgreSQL Cloud.

01 Penyimpanan permanen

Memahami keterbatasan variabel array di memori saat server *restart* dan peran *database* relasional.

02 PostgreSQL di Render Cloud

Membuat *instance* PostgreSQL siap pakai secara daring melalui layanan *Database as a Service* (DaaS) gratis.

03 Anatomi *Connection String*

Mengenal bagian-bagian alamat koneksi (`user` , `password` , `host` , dan `namadb`).

04 Keamanan berkas `.env`

Memisahkan kata sandi *database* dari kode sumber agar tidak bocor saat diunggah ke GitHub.

05 Integrasi `dotenv` & Lab 2

Memasang paket `dotenv` , menyalin *External Database URL*, dan membaca `process.env` di dalam `app.js` .

01 • KEBUTUHAN PENYIMPANAN PERMANEN

Keterbatasan menyimpan data di dalam array atau variabel

Pada Hari 1, kita menyimpan daftar karyawan di dalam variabel array JavaScript. Cara tersebut hanya bersifat sementara:

VARIABEL / ARRAY DI MEMORI (RAM)

Data hilang saat server berhenti

Setiap kali kita menekan `Ctrl + C` atau menjalankan ulang aplikasi Node.js, seluruh data baru yang ditambahkan pengguna akan hilang dan kembali ke isi awal.

DATABASE RELASIONAL • POSTGRESQL

Data tersimpan permanen dan terstruktur

Data disimpan di dalam tabel permanen pada media penyimpanan, tetap utuh meskipun server aplikasi dimatikan, serta mendukung pencarian dan relasi antartabel.

> Aplikasi web modern selalu memisahkan proses pengolahan logika (Node.js) dari tempat penyimpanan data permanen (*database*).

01 • MEMBUAT DATABASE POSTGRESQL

Mengenal PostgreSQL dan layanan *Database as a Service*

Kita tidak perlu menginstal server *database* yang berat di komputer masing-masing karena tersedia layanan *cloud* siap pakai:

DATABASE RELASIONAL OPEN-SOURCE

PostgreSQL

Sistem manajemen basis data relasional (SQL) tingkat lanjut yang banyak dipakai di industri modern.

- Menyimpan data secara rapi di dalam struktur baris dan kolom tabel.
- Mendukung tipe data lengkap, relasi antartabel, dan konsistensi transaksi.
- Terintegrasi sangat baik dengan ekosistem Node.js dan Prisma ORM.

Relasional (SQL) **Open-Source** **Stabil & Andal**

DAAS • DATABASE AS A SERVICE

Render.com (*Free Tier*)

Platform *cloud* yang menyediakan *instance* PostgreSQL secara daring hanya dalam hitungan menit.

- Tidak membebani memori atau ruang penyimpanan lokal di laptop peserta.
- Menyediakan paket gratis (*Free Tier*) yang dapat diakses dari mana saja lewat internet.
- Cukup memilih lokasi server terdekat (contoh: **Singapore**) lalu menyalin alamat koneksinya.

Render Cloud **Region Singapore** **Tanpa Instalasi Lokal**

02 • MENYALIN CONNECTION STRING

Membedah anatomi *Connection String* (URI Database)

Setiap aplikasi Node.js memerlukan sebuah alamat lengkap beserta kunci akses untuk terhubung ke *database*, yang disebut **Connection String**:

FORMAT UMUM CONNECTION STRING

```
postgresql://user:password@host/namadb

// Contoh External Database URL dari Render:
postgresql://inix_user:KunciRahasia99@
dpg-xyz123.singapore-postgres.render.com/db_pegawai
```

Seluruh informasi login dan lokasi server dirangkum dalam satu baris URI.

01 `postgresql://`

Menunjukkan protokol jenis *database* yang digunakan oleh aplikasi.

02 `user:password@`

Nama pengguna dan kata sandi rahasia yang dibuat otomatis oleh Render.

03 `host/namadb`

Alamat domain server *cloud* Render diikuti nama *database* tujuan.

02 • KETELITIAN MEMILIH URL DI RENDER

Perbedaan *Internal* dan *External Database URL*

Pada halaman informasi *database* di Render, Anda akan melihat dua jenis URL koneksi. Pastikan menyalin bagian yang tepat:

JENIS URL DI RENDER	CIRI ALAMAT HOST	KAPAN DIGUNAKAN?
`Internal Database URL`	Pendek, tanpa akhiran domain `.render.com`	Hanya untuk aplikasi yang sudah berjalan di dalam jaringan internal Render pada region yang sama.
`External Database URL`	Lengkap dengan `.singapore-postgres.render.com`	**Digunakan pada pelatihan ini** agar aplikasi Node.js di laptop Anda dapat menghubungi *database* di *cloud*.

> **Catatan penting untuk Lab 2:** Selalu salin bagian **External Database URL** agar koneksi dari komputer lokal Anda dapat terhubung ke Render.

03 • MENYIMPAN KONFIGURASI DATABASE DI .ENV

Mengapa *Connection String* harus disimpan di berkas .env?

Connection String mengandung *username* dan *password database* Anda. Menuliskannya langsung di dalam kode utama sangat berbahaya:

01 RISIKO KEBOCORAN

Kode diunggah ke GitHub

Jika URL *database* tertulis langsung di kode lalu diunggah ke repositori publik, pihak luar dapat mencuri akses *database* Anda.

> Jangan pernah menaruh kata sandi di dalam `app.js`.

02 BERKAS RAHASIA

Disimpan di .env

Seluruh kunci rahasia, URL *database*, dan nomor port disimpan di dalam berkas khusus bernama `.env` di folder terluar proyek.

> Diletakkan sejajar dengan berkas `package.json`.

03 PERLINDUNGAN GIT

Dicegah oleh `.gitignore`

Berkas `.env` didaftarkan ke dalam `.gitignore` sehingga tidak akan ikut terunggah ke GitHub saat kita menjalankan `git push`.

> Kode publik aman, rahasia tetap di komputer lokal.

03 • MEMBACA BERKAS .ENV DI NODE.JS

Menghubungkan berkas .env ke app.js dengan dotenv

Node.js membaca isi berkas `.env` menggunakan paket bernama `dotenv` lalu menyediakannya melalui objek `process.env` :

ISI `.ENV` & PEMANGGILAN DI `APP.JS`

```
// 1. Isi berkas .env:
PORT=3000
DATABASE_URL="postgresql://user:pass@host/namadb"

// 2. Baris paling atas di app.js:
require('dotenv').config();
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;
```

Letakkan `require('dotenv').config()` di baris pertama sebelum kode lainnya.

01 `npm install dotenv`

Memasang pustaka pembaca variabel lingkungan (*environment variables*) ke dalam proyek Express.

02 `require('dotenv').config()`

Memuat seluruh pasangan kunci dan nilai dari berkas `.env` saat aplikasi dinyalakan.

03 `process.env.PORT`

Mengambil nilai variabel `PORT` dari berkas `.env`, atau menggunakan angka `3000` sebagai cadangan.

03 • STANDAR KERAPIAN PROYEK

Aturan penulisan berkas `.env` dan perlindungan `.gitignore`

Perhatikan tiga aturan penulisan berikut agar variabel di dalam `.env` terbaca tanpa kendala oleh Node.js maupun Prisma:

01 **Nama variabel HURUF_KAPITAL**

Tulis nama variabel seperti `PORT=3000` dan `DATABASE_URL="..."` tanpa spasi di kiri atau kanan tanda `=`.

02 **Apit URL dengan tanda kutip ganda**

Selalu bungkus nilai *Connection String* dengan tanda kutip ganda (`"postgresql://..."`) agar karakter khusus terbaca utuh.

03 **Daftarkan di dalam `.gitignore`**

Pastikan nama berkas `.env` dan folder `node_modules/` tercatat di dalam berkas `.gitignore`.

CONTOH BERKAS `.GITIGNORE`

```
# Abaikan folder pustaka npm
node_modules/

# Abaikan berkas rahasia environment
.env
```

Dengan pengaturan ini, kata sandi *database* Anda aman dari risiko terunggah ke Git.

SIMULASI INTERAKTIF • ALUR KONFIGURASI CLOUD

Tiga tahap menyiapkan PostgreSQL Cloud hingga terbaca di Node.js

1. Buat Instance di Render
2. Simpan di Berkas .env
3. Baca lewat dotenv

TAHAP 1 • DASHBOARD RENDER.COM

Membuat instance PostgreSQL baru di region Singapore (Free Tier)

Setelah memilih New -> PostgreSQL dan menekan Create Database, tunggu hingga status berubah menjadi Available, lalu salin External Database URL.

INFORMASI INSTANCE DI RENDER

```
Name     : inix-modern-web-db
Region   : Singapore (Southeast Asia)
Plan     : Free ($0 / month)
Status   : Available

External Database URL:
postgresql://inix_user:xYz987@dpg-123.singapore-
postgres.render.com/inix_db
```

Salin seluruh teks yang diawali dengan `postgresql://` pada bagian External Database URL.

SIMULASI INTERAKTIF • ALUR KONFIGURASI CLOUD

Tiga tahap menyiapkan PostgreSQL Cloud hingga terbaca di Node.js

1. Buat Instance di Render | **2. Simpan di Berkas .env** | 3. Baca lewat dotenv

TAHAP 2 • BERKAS .ENV DI VS CODE

Menyimpan External Database URL ke dalam berkas .env

Buat berkas .env sejajar dengan package.json, lalu tempelkan URL yang baru disalin ke dalam variabel bernama DATABASE_URL.

ISI BERKAS .ENV

```
PORT=3000
DATABASE_URL="postgresql://inix_user:xYz987@dpg-
123.singapore-postgres.render.com/inix_db"
```

Nama variabel `DATABASE_URL` ini yang nantinya juga akan dibaca otomatis oleh Prisma ORM pada Materi 3.

SIMULASI INTERAKTIF • ALUR KONFIGURASI CLOUD

Tiga tahap menyiapkan PostgreSQL Cloud hingga terbaca di Node.js

1. Buat Instance di Render
2. Simpan di Berkas .env
3. Baca lewat dotenv

TAHAP 3 • SISI SERVER (APP.JS)

Mengaktifkan dotenv agar process.env siap digunakan

Saat aplikasi dijalankan, require('dotenv').config() memuat isi .env ke dalam memori proses Node.js tanpa menampilkannya di kode utama.

BAGIAN ATAS BERKAS APP.JS

```
require('dotenv').config();
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Server berjalan di
http://localhost:${port}`);
});
```

Kredensial *database* tetap rahasia di dalam `.env` sementara kode `app.js` tetap bersih.

PERSIAPAN LAB 2 • STRUKTUR FOLDER PROYEK

Letak berkas .env dan pembaruan pada app.js

Pastikan berkas `.env` dibuat persis di direktori paling luar (sejajar dengan `package.json` dan `app.js`):

POSISI BERKAS `.ENV` & KODE `APP.JS`

```
project-express/
├── node_modules/
├── views/
├── .env            <-- Buat berkas baru di sini
├── app.js          <-- Tambahkan dotenv di baris ke-1
└── package.json

// Baris teratas di dalam app.js:
require('dotenv').config();
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;
```

01 Diawali tanda titik (.env)

Pastikan nama berkas persis `.env` (bukan `env.txt` atau `nama.env`) agar dikenali otomatis oleh `dotenv`.

02 Nilai cadangan || 3000

Ekspresi `process.env.PORT || 3000` menjaga server tetap berjalan di port 3000 bila variabel `PORT` belum diisi.

03 Siap untuk Prisma ORM

Variabel `DATABASE_URL` di dalam `.env` akan langsung dibaca oleh Prisma pada sesi berikutnya.

PRAKTIK MANDIRI • LAB 2 HARI 2

Saatnya praktik: menyiapkan PostgreSQL di Render & berkas .env

Ikuti langkah **Lab 2: Setup Database Render** pada proyek Anda:

1. Kunjungi `https://render.com`, buat akun/masuk, lalu klik **New** -> **PostgreSQL**.
2. Isi nama *database*, pilih lokasi **Singapore**, pilih paket **Free**, klik **Create Database**, dan tunggu berstatus *Available*.
3. Salin **External Database URL** (`postgresql://...`), lalu buat berkas `.env` sejajar dengan `package.json`.
4. Pasang paket `dotenv` di terminal, isi `.env`, dan perbarui bagian atas `app.js`:

```
npm install dotenv
```

Isi `PORT=3000` dan `DATABASE_URL="External_URL_Anda"` di dalam `.env`, lalu tambahkan `require('dotenv').config();` di baris pertama `app.js`.

TARGET LAB 2

Database cloud aktif & .env siap

Instance PostgreSQL di Render berstatus *Available*, *External Database URL* tersimpan aman di dalam `.env`, dan server Express berjalan normal menggunakan `process.env.PORT`.

SESI BERIKUTNYA

Pada **Materi 3 Hari 2**, kita akan menghubungkan aplikasi Node.js ke *database* Render menggunakan **Prisma ORM** dan menjalankan migrasi tabel pertama.

HARI 2 • MATERI 3

Koneksi ke *database* dengan Prisma ORM

Menghubungkan Node.js ke PostgreSQL menggunakan *Object-Relational Mapping* (ORM), merancang model di `schema.prisma`, dan membentuk tabel fisik melalui proses migrasi.

Kode: TX-BE01 **50 menit Materi 3 & Lab 3**

POKOK BAHASAN SESI INI

01 Konsep ORM dan keunggulan Prisma di Node.js

02 Peran paket `prisma` (CLI) dan `@prisma/client`

03 Struktur berkas konfigurasi `prisma/schema.prisma`

04 Merancang model `Barang` beserta tipe datanya

05 Menjalankan *migration* ke Render Cloud & praktik Lab 3

PETA BELAJAR

Lima langkah menghubungkan aplikasi ke PostgreSQL

Mengubah definisi model di kode proyek menjadi tabel nyata di dalam *database* Render Cloud.

01 Konsep ORM (Prisma)

Berinteraksi dengan tabel *database* menggunakan metode objek JavaScript tanpa menulis perintah SQL mentah.

02 Inisialisasi Prisma

Memasang paket `prisma` dan `@prisma/client`, lalu menjalankan `npx prisma init`.

03 Berkas `schema.prisma`

Memahami blok `generator client`, `datasource db`, dan pembacaan `env("DATABASE_URL")`.

04 Menyusun model `Barang`

Mendefinisikan kolom `id`, `nama`, `harga`, `stok`, dan `createdAt` beserta aturan otomatisnya.

05 Proses *Migration* & Lab 3

Menjalankan `npx prisma migrate dev` untuk mencetak tabel fisik di PostgreSQL Render.

01 • MENGGUNAKAN PRISMA SEBAGAI ORM

Perbandingan perintah SQL mentah dengan pendekatan ORM

Untuk memberi perintah seperti `SELECT` atau `INSERT` dari Node.js ke PostgreSQL, cara modern dan lebih aman adalah menggunakan **ORM (Object-Relational Mapping)**:

CARA KONVENSIONAL • RAW SQL QUERY

Menulis kalimat SQL manual di dalam string

Contoh: `SELECT * FROM "Barang" WHERE stok > 0;`. Rentan salah ketik nama kolom, kurang menyatu dengan objek JavaScript, dan rawan celah keamanan bila parameter tidak disaring.

CARA MODERN • PRISMA ORM

Memanggil metode objek JavaScript

Contoh: `prisma.barang.findMany()`. Lebih ringkas, dilengkapi saran pengetikan otomatis (*auto-complete*) di VS Code, dan aman dari celah *SQL Injection*.

> Prisma bertindak sebagai jembatan penerjemah antara kode JavaScript di Express dan tabel relasional di PostgreSQL.

01 • ARSITEKTUR PAKET PRISMA

Membedakan peran `prisma` (CLI) dan `@prisma/client`

Saat mengintegrasikan Prisma ke proyek Node.js, kita memasang dua paket yang memiliki tugas berbeda:

ALAT BANTU PENGEMBANG • --SAVE-DEV

Paket `prisma` (CLI)

Alat baris perintah (*Command Line Interface*) yang dijalankan di terminal saat merancang dan memperbarui struktur tabel.

- Dipasang dengan perintah `npm install prisma --save-dev`.
- Menjalankan inisialisasi konfigurasi awal (`npx prisma init`).
- Mengeksekusi migrasi skema tabel ke *database* (`npx prisma migrate dev`).

npx prisma init | **npx prisma migrate** | **Dev Dependency**

PUSTAKA APLIKASI • RUNTIME

Paket `@prisma/client`

Pustaka JavaScript yang diimpor di dalam `app.js` untuk membaca, menambah, mengubah, dan menghapus baris data.

- Dipasang dengan perintah `npm install @prisma/client`.
- Menyediakan kelas `PrismaClient` yang menyesuaikan bentuk model tabel kita.
- Digunakan langsung di dalam rute-rute Express pada Materi 4 nanti.

PrismaClient | **findMany()** | **create() / update()**

02 • MEMBUAT SCHEMA MODEL

Inisialisasi Prisma dan isi awal `prisma/schema.prisma`

Perintah `npx prisma init` otomatis membuat folder `prisma/` berisi berkas pengaturan skema bernama `schema.prisma`:

BAGIAN ATAS `PRISMA/SCHEMA.PRISMA`

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

Pastikan nilai `provider` pada blok `datasource db` adalah `"postgresql"`.

01 **Blok** `generator client`

Memberi tahu Prisma agar menghasilkan pustaka `@prisma/client` khusus untuk lingkungan JavaScript/Node.js.

02 `provider = "postgresql"`

Menentukan bahwa jenis *database* tujuan kita adalah PostgreSQL sesuai *instance* di Render.

03 `url = env("DATABASE_URL")`

Membaca alamat koneksi *External Database URL* yang sudah kita simpan di dalam berkas `.env` pada Lab 2.

02 • MENDEFINISIKAN STRUKTUR TABEL

Menuliskan model Barang di dalam schema.prisma

Di bagian bawah berkas `prisma/schema.prisma`, kita mendefinisikan struktur tabel beserta nama kolom dan tipe datanya:

MODEL `BARANG` DI `PRISMA/SCHEMA.PRISMA`

```
model Barang {
  id        Int      @id @default(autoincrement())
  nama      String
  harga     Int
  stok      Int
  createdAt DateTime @default(now())
}
```

Satu blok `model` mewakili satu tabel di dalam *database* PostgreSQL.

01 Kolom `id` (Kunci Utama)

Bertipe angka bulat (`Int`), menjadi *Primary Key* (`@id`) yang nilainya bertambah otomatis.

02 Kolom `nama`, `harga`, `stok`

Menyimpan teks nama barang (`String`) serta nilai harga satuan dan jumlah stok barang (`Int`).

03 Kolom `createdAt`

Mencatat tanggal dan waktu saat baris barang dibuat secara otomatis (`@default(now())`).

02 • KAMUS SINTAKS SCHEMA PRISMA

Arti tipe data dan atribut khusus pada `schema.prisma`

Setiap baris di dalam model terdiri atas tiga kolom: nama atribut, tipe data, serta pengatur tambahan yang diawali tanda `@`:

SINTAKS PRISMA	KATEGORI	FUNGSI PADA TABEL DATABASE POSTGRESQL
`String` / `Int` / `DateTime`	Tipe Data Kolom	Menentukan jenis nilai kolom: teks (`TEXT`), angka bulat (`INTEGER`), atau waktu (`TIMESTAMP`).
`@id`	Atribut Kunci Utama	Menetapkan kolom tersebut sebagai *Primary Key* (identitas unik untuk setiap baris barang).
`@default(autoincrement())`	Atribut Nilai Awal	Mengisi nomor urut otomatis (1, 2, 3, dst.) setiap kali baris barang baru disimpan.
`@default(now())`	Atribut Nilai Awal	Mengisi stempel waktu saat ini secara otomatis ketika baris data pertama kali ditambahkan.

> **Kemudahan saat input data:** Karena `id` dan `createdAt` terisi otomatis, formulir tambah barang nantinya cukup meminta input `nama`, `harga`, dan `stok`.

03 • MIGRATION

Bagaimana proses *migration* membentuk tabel di *cloud*?

Migration adalah proses penerjemahan dari berkas `schema.prisma` menjadi perintah SQL yang dijalankan di PostgreSQL Render:

01 BACA SKEMA

Periksa perubahan

Prisma membaca isi `schema.prisma` lalu membandingkannya dengan struktur tabel yang ada di *database* Render saat ini.

> Mendeteksi adanya model baru bernama `Barang`.

02 CATAT VERSI SQL

Riwayat perubahan

Prisma membuat berkas `migration.sql` di dalam folder `prisma/migrations/` untuk melacak riwayat perubahan tabel (*versioning*).

> Berisi perintah SQL `CREATE TABLE "Barang"`.

03 CETAK TABEL & CLIENT

Tabel nyata terbentuk

Perintah SQL dijalankan di Render hingga tabel fisik terbentuk, lalu `@prisma/client` diperbarui agar mengenali model `Barang`.

> Objek `prisma.barang` siap dipanggil di `app.js`.

03 • MENJALANKAN MIGRASI

Menjalankan `npx prisma migrate dev --name init_barang`

Setelah model `Barang` disimpan di `schema.prisma` dan `DATABASE_URL` siap di `.env`, kita jalankan perintah migrasi di terminal:

01 `npx prisma migrate dev`

Membandingkan skema, menjalankan SQL ke *database*, dan menghasilkan ulang kode *Prisma Client*.

02 `--name init_barang`

Memberikan nama penanda pada folder riwayat migrasi pertama agar mudah dikenali.

03 **Folder** `prisma/migrations/`

Otomatis dibuat di proyek Anda sebagai catatan riwayat versi skema *database*.

PERINTAH & KELUARAN DI TERMINAL

```
npx prisma migrate dev --name init_barang

# Keluaran saat berhasil:
Applying migration `20261031_init_barang`
Your database is now in sync with your schema.
✔ Generated Prisma Client
```

Pesan *"Your database is now in sync"* menandakan tabel `Barang` sudah terbentuk di Render.

SIMULASI INTERAKTIF • ALUR KERJA PRISMA

Transformasi dari `schema.prisma` hingga siap dipanggil di Node.js

1. Model di schema.prisma | 2. SQL Hasil Migrasi | 3. Objek Prisma Client

TAHAP 1 • DEFINISI SKEMA

Kita menulis model Barang dengan sintaks Prisma yang ringkas

Kita cukup menuliskan nama kolom, tipe data, dan aturan @id atau @default di dalam berkas prisma/schema.prisma.

BERKAS PRISMA/SCHEMA.PRISMA

```
model Barang {
  id        Int      @id @default(autoincrement())
  nama      String
  harga     Int
  stok      Int
  createdAt DateTime @default(now())
}
```

Mudah dibaca dan dikelola di dalam folder `prisma/`.

SIMULASI INTERAKTIF • ALUR KERJA PRISMA

Transformasi dari `schema.prisma` hingga siap dipanggil di Node.js

1. Model di schema.prisma | **2. SQL Hasil Migrasi** | 3. Objek Prisma Client

TAHAP 2 • PENERJEMAHAN MIGRASI

Prisma menerjemahkan model menjadi perintah SQL PostgreSQL

Saat npx prisma migrate dev dijalankan, Prisma membuat berkas migration.sql dan mengeksekusinya ke server PostgreSQL Render.

ISI PRISMA/MIGRATIONS/.../MIGRATION.SQL

```
CREATE TABLE "Barang" (
  "id" SERIAL NOT NULL,
  "nama" TEXT NOT NULL,
  "harga" INTEGER NOT NULL,
  "stok" INTEGER NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  CONSTRAINT "Barang_pkey" PRIMARY KEY ("id")
);
```

Kita tidak perlu mengetik sintaks `CREATE TABLE` ini secara manual.

SIMULASI INTERAKTIF • ALUR KERJA PRISMA

Transformasi dari schema.prisma hingga siap dipanggil di Node.js

1. Model di schema.prisma | 2. SQL Hasil Migrasi | **3. Objek Prisma Client**

TAHAP 3 • SIAP DIPAKAI DI EXPRESS

Prisma Client menyediakan properti prisma.barang di JavaScript

Setelah migrasi selesai, Prisma otomatis memperbarui @prisma/client sehingga kita bisa mengakses tabel Barang langsung dari app.js.

PEMANGGILAN DI DALAM APP.JS

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

// Mengambil seluruh baris dari tabel Barang:
const semuaBarang = await prisma.barang.findMany();
```

Perhatikan bahwa model `Barang` diakses dengan huruf awal kecil menjadi `prisma.barang`.

TIPS PENGEMBANG • PEMERIKSAAN VISUAL

Melihat tabel di *cloud* dengan `npx prisma studio`

Selain lewat kode, Prisma menyediakan antarmuka tabel visual di peramban untuk memastikan tabel `Barang` sudah terbentuk:

PERINTAH PEMERIKSAAN VISUAL DI TERMINAL

```
npx prisma studio

# Otomatis membuka peramban pada alamat:
# http://localhost:5555
```

Tekan `Ctrl + C` di terminal jika sudah selesai melihat Prisma Studio.

01 Dasbor tabel bawaan Prisma

Menampilkan daftar tabel yang ada di *database* Render Anda tanpa perlu menginstal aplikasi tambahan seperti pgAdmin.

02 Cek kolom dan isi data

Anda dapat melihat kolom `id`, `nama`, `harga`, `stok`, dan `createdAt` secara langsung di layar.

03 Memudahkan pemantauan CRUD

Sangat berguna pada sesi berikutnya untuk memeriksa data barang yang berhasil disimpan atau diubah.

PRAKTIK MANDIRI • LAB 3 HARI 2

Saatnya praktik: instalasi Prisma dan migrasi model Barang

Ikuti langkah **Lab 3: Prisma ORM** pada proyek Express Anda:

1. Pasang paket di terminal: `npm install prisma --save-dev` lalu `npm install @prisma/client`.
2. Inisialisasi Prisma dengan menjalankan perintah `npx prisma init`.
3. Buka `prisma/schema.prisma`, pastikan `provider = "postgresql"`, dan tambahkan `model Barang { ... }` di bagian bawah.
4. Jalankan perintah migrasi berikut untuk mencetak tabel fisik di Render:

```
npx prisma migrate dev --name init_barang
```

Proses ini akan mengunduh *Prisma Engine*, membandingkan skema, dan membuat tabel `Barang` secara otomatis di *database* Render.com Anda.

TARGET LAB 3

Tabel Barang terbentuk di Render

Terminal menampilkan pesan `Your database is now in sync with your schema` dan `Generated Prisma Client`, disertai munculnya folder `prisma/migrations/`.

SESI BERIKUTNYA

Setelah tabel `Barang` siap di *database*, pada **Materi 4 Hari 2** kita akan membangun fitur **CRUD** lengkap (*Create, Read, Update, Delete*).

HARI 2 • MATERI 4

Membangun fitur CRUD dengan Express dan Prisma

Menerapkan operasi *Create*, *Read*, *Update*, dan *Delete* secara utuh dari formulir dan tabel EJS langsung ke *database* PostgreSQL Cloud di Render.

Kode: TX-BE01 **80 menit Materi 4 & Lab 4**

POKOK BAHASAN SESI INI

01 Empat operasi dasar CRUD dan metode Prisma Client

02 Operasi *Read*: mengambil daftar barang dengan `findMany()`

03 Operasi *Create*: menyimpan input dengan `create()` & `parseInt()`

04 Operasi *Update*: `findUnique()` dan `update()`

05 Operasi *Delete* dengan konfirmasi & praktik Lab 4

PETA BELAJAR

Empat pilar pengelolaan data di aplikasi web

Menghubungkan antarmuka EJS dan Bootstrap dengan tabel `Barang` di PostgreSQL Render menggunakan Prisma Client.

01 Read (`findMany`)

Menarik seluruh baris dari tabel `Barang` dan menampilkannya di halaman `views/barang.ejs`.

02 Create (`create`)

Menerima kiriman formulir dari `views/tambah-barang.ejs` lalu menyimpannya ke dalam *database*.

03 Update (`findUnique` & `update`)

Menampilkan formulir berisi nilai lama berdasarkan `id`, lalu memperbarui datanya di *database*.

04 Delete (`delete`)

Menghapus suatu baris tabel berdasarkan *Primary Key* disertai konfirmasi `confirm()` di peramban.

05 Praktik Lab 4: CRUD Operation

Membangun halaman manajemen barang (`/barang` dan `/barang/tambah`) yang tersimpan permanen di Render Cloud.

IKHTISAR CRUD • METODE PRISMA CLIENT

Pemetaan operasi CRUD ke metode `prisma.barang`

Setelah memanggil `const prisma = new PrismaClient()` di `app.js`, seluruh operasi tabel dilakukan lewat fungsi `async / await`:

OPERASI CRUD	METODE PRISMA CLIENT	PERAN DALAM APLIKASI MANAJEMEN BARANG
Create (Tambah)	`await prisma.barang.create({ data })`	Menyimpan data barang baru dari `req.body` ke dalam tabel `Barang`.
Read (Baca)	`await prisma.barang.findMany()`	Mengambil semua baris barang sebagai *Array of Objects* untuk tabel EJS.
Update (Ubah)	`await prisma.barang.update({ where, data })`	Memperbarui kolom `nama`, `harga`, atau `stok` pada baris ber-`id` tertentu.
Delete (Hapus)	`await prisma.barang.delete({ where })`	Menghapus satu baris barang dari *database* berdasarkan *Primary Key* (`id`).

> **Mengapa memakai async dan await?** Komunikasi ke *database* di *cloud* memerlukan waktu jaringan, sehingga Express harus menunggu perintah selesai sebelum merender halaman.

01 • READ (MENAMPILKAN DATA)

Menarik dan menampilkan daftar barang dengan `findMany()`

Pada rute `GET /barang`, kita memanggil `prisma.barang.findMany()` lalu mengirimkan hasilnya ke `views/barang.ejs`:

RUTE READ DI DALAM `APP.JS`

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

// READ: Ambil semua baris dari tabel Barang
app.get('/barang', async (req, res) => {
  const barangs = await prisma.barang.findMany();
  res.render('barang', {
    listBarang: barangs
  });
});
```

Kata kunci `async` pada fungsi penangan wajib berpasangan dengan `await`.

01 `new PrismaClient()`

Membuka hubungan kerja ke PostgreSQL Render menggunakan konfigurasi `DATABASE_URL`.

02 `await prisma.barang.findMany()`

Menarik seluruh baris dari tabel `Barang` dalam bentuk *Array of Objects*.

03 `res.render('barang', ...)`

Meneruskan data dari *cloud* ke berkas `views/barang.ejs` dengan nama `listBarang`.

02 • CREATE (MENYIMPAN DATA)

Alur dua rute untuk menambahkan barang baru

Fitur tambah barang melibatkan dua rute dengan jalur alamat yang sama (`/barang/tambah`), tetapi berbeda metode HTTP-nya:

LANGKAH PERTAMA • METODE GET

GET /barang/tambah

Menampilkan halaman formulir kosong kepada pengguna saat tombol "Tambah Barang" diklik.

- Cukup memanggil `res.render('tambah-barang')` .
- Menyajikan kotak input untuk mengisi `nama` , `harga` , dan `stok` .
- Formulir memiliki pengaturan `action="/barang/tambah"` `method="POST"` .

app.get() **res.render()** **Tampilkan Form**

LANGKAH KEDUA • METODE POST

POST /barang/tambah

Menerima kiriman data dari formulir HTML, menyimpannya ke PostgreSQL, lalu mengalihkan halaman.

- Membaca input `nama` , `harga` , dan `stok` dari `req.body` .
- Menjalankan `await prisma.barang.create({ data: ... })` .
- Mengembalikan pengguna ke halaman tabel utama dengan `res.redirect('/barang')` .

app.post() **prisma.barang.create()** **res.redirect()**

02 • KETELITIAN TIPE DATA PADA CREATE

Menyimpan data dengan `create()` dan konversi `parseInt()`

Semua nilai dari formulir HTML (`req.body`) diterima sebagai teks (`String`), sehingga kolom angka wajib diubah dengan `parseInt()` :

RUTE CREATE DI DALAM `APP.JS`

```
// CREATE (Form View)
app.get('/barang/tambah', (req, res) => {
  res.render('tambah-barang');
});

// CREATE (Action POST)
app.post('/barang/tambah', async (req, res) => {
  const { nama, harga, stok } = req.body;
  await prisma.barang.create({
    data: {
      nama: nama,
      harga: parseInt(harga),
      stok: parseInt(stok)
    }
  });
  res.redirect('/barang'); // Kembali ke tabel utama
});
```

01 **Mengapa wajib `parseInt()`?**

Di `schema.prisma` , kolom `harga` dan `stok` bertipe `Int` . Fungsi `parseInt()` mengubah teks `"15000"` menjadi angka `15000` .

02 **Properti `data: { ... }`**

Memasukkan nilai untuk kolom `nama` , `harga` , dan `stok` ke baris baru di tabel `Barang` .

03 **`res.redirect('/barang')`**

Mengalihkan peramban kembali ke `/barang` agar barang yang baru disimpan langsung terlihat di tabel.

03 • UPDATE (MENGEDIT DATA)

Dua langkah memperbarui data dengan `findUnique` dan `update`

Alur *update* melibatkan pengambilan satu baris spesifik terlebih dahulu sebelum mengirimkan perubahan datanya:

01 **Cari data lama (`findUnique`)**

Mengambil satu baris barang berdasarkan `id` agar formulir edit langsung terisi nilai lama pada atribut `value="..."`.

02 **Simpan perubahan (`update`)**

Menerima kiriman `POST` dari formulir edit lalu memperbarui baris yang cocok pada klausa `where: { id }`.

03 **Parameter URL `req.params.id`**

Nilai `:id` dari alamat URL juga diubah ke angka bulat menggunakan `parseInt(req.params.id)`.

POLA RUTE UPDATE DI `APP.JS`

```
// 1. Tampilkan form berisi nilai lama
app.get('/barang/edit/:id', async (req, res) => {
  const barang = await prisma.barang.findUnique({
    where: { id: parseInt(req.params.id) }
  });
  res.render('edit-barang', { b: barang });
});

// 2. Perbarui data di database
app.post('/barang/edit/:id', async (req, res) =>
{
  const { nama, harga, stok } = req.body;
  await prisma.barang.update({
    where: { id: parseInt(req.params.id) },
    data: { nama, harga: parseInt(harga), stok:
parseInt(stok) }
  });
  res.redirect('/barang');
});
```

04 • DELETE (MENGHAPUS DATA)

Menghapus baris berdasarkan ID dan konfirmasi `confirm()`

Operasi *delete* menghapus satu baris berdasarkan *Primary Key* (`id`), disertai konfirmasi di peramban agar tidak terhapus secara tidak sengaja:

TOMBOL DI EJS & RUTE HAPUS DI `app.js`

```
<!-- Tombol Hapus di dalam tabel EJS -->
<form action="/barang/hapus/<%= b.id %>" method="POST"
      onsubmit="return confirm('Yakin hapus barang ini?')">
  <button type="submit" class="btn btn-danger btn-sm">
    Hapus
  </button>
</form>

// Rute Hapus di dalam app.js:
app.post('/barang/hapus/:id', async (req, res) => {
  await prisma.barang.delete({
    where: { id: parseInt(req.params.id) }
  });
  res.redirect('/barang');
});
```

01 **Fungsi JavaScript `confirm()`**

Menampilkan kotak dialog konfirmasi di peramban. Jika pengguna memilih *Cancel*, formulir batal dikirim.

02 **`prisma.barang.delete({ where })`**

Menghapus baris pada tabel `Barang` yang memiliki nilai `id` sesuai parameter URL.

03 **Segarkan halaman tabel**

`res.redirect('/barang')` memuat ulang halaman daftar barang sehingga baris yang dihapus langsung hilang dari layar.

SIMULASI INTERAKTIF • ALUR KERJA CRUD

Perjalanan data dari formulir EJS hingga tersimpan di *cloud*

1. Tambah Barang (Create) | 2. Tampilkan Tabel (Read) | 3. Ubah & Hapus (Update/Delete)

TAHAP 1 • CREATE (POST /BARANG/TAMBAH)

Pengguna mengisi formulir dan menekan tombol Simpan

Express menerima req.body, mengubah harga dan stok menjadi angka bulat dengan parseInt(), lalu memanggil prisma.barang.create() ke Render.

PROSES PENYIMPANAN DI SERVER

```
// Input form: nama="Monitor 24 Inci", harga="1850000",
stok="12"
await prisma.barang.create({
  data: {
    nama: 'Monitor 24 Inci',
    harga: parseInt('1850000'), // -> 1850000
    stok: parseInt('12')        // -> 12
  }
});
res.redirect('/barang');
```

Baris baru tersimpan permanen di PostgreSQL Render dengan `id` otomatis.

SIMULASI INTERAKTIF • ALUR KERJA CRUD

Perjalanan data dari formulir EJS hingga tersimpan di *cloud*

1. Tambah Barang (Create) | **2. Tampilkan Tabel (Read)** | 3. Ubah & Hapus (Update/Delete)

TAHAP 2 • READ (GET /BARANG)

Halaman /barang menarik seluruh baris dari PostgreSQL Cloud

Setelah dialihkan oleh res.redirect('/barang'), rute GET /barang memanggil prisma.barang.findMany() dan mengirimkannya ke barang.ejs.

DATA YANG DIKIRIM KE VIEWS/BARANG.EJS

```
const barangs = await prisma.barang.findMany();
// Isi barangs dari database Render:
// [
//   { id: 1, nama: 'Monitor 24 Inci', harga: 1850000,
stok: 12, ... }
// ]
res.render('barang', { listBarang: barangs });
```

Meskipun server Node.js dimatikan dan dinyalakan lagi, data ini tetap utuh.

SIMULASI INTERAKTIF • ALUR KERJA CRUD

Perjalanan data dari formulir EJS hingga tersimpan di *cloud*

1. Tambah Barang (Create) | 2. Tampilkan Tabel (Read) | **3. Ubah & Hapus (Update/Delete)**

TAHAP 3 • UPDATE & DELETE BERDASARKAN ID

Memilih satu baris spesifik menggunakan klausa where: { id }

Baik saat mengambil satu barang (findUnique), mengubah (update), maupun menghapus (delete), kita selalu menyertakan id barang pada properti where.

PENCOCOKAN PRIMARY KEY DI PRISMA

```
// Mengubah stok barang ber-ID 1 menjadi 20:
await prisma.barang.update({
  where: { id: 1 },
  data: { stok: 20 }
});

// Menghapus barang ber-ID 1:
await prisma.barang.delete({ where: { id: 1 } });
```

Klausa `where: { id }` memastikan perubahan hanya mengenai satu baris sasaran.

PERSIAPAN LAB 4 • HALAMAN DAFTAR BARANG

Struktur berkas `views/barang.ejs`

Menampilkan tombol menuju halaman tambah barang serta tabel daftar barang dari *database*:

BERKAS `VIEWS/BARANG.EJS`

```
<body class="container mt-5">
  <h2>Manajemen Barang</h2>
  <a href="/barang/tambah" class="btn btn-primary mb-3">
    Tambah Barang
  </a>
  <table class="table table-bordered">
    <thead>
      <tr><th>ID</th><th>Nama</th><th>Harga</th><th>Stok</th>
</tr>
    </thead>
    <tbody>
      <% listBarang.forEach(b => { %>
        <tr>
          <td><%= b.id %></td>
          <td><%= b.nama %></td>
          <td><%= b.harga %></td>
          <td><%= b.stok %></td>
        </tr>
      <% }) %>
    </tbody>
  </table>
</body>
```

01 **Tombol** `Tambah Barang`

Tautan `<a href="/barang/tambah">` mengarahkan pengguna ke halaman formulir tambah barang.

02 **Iterasi** `listBarang.forEach`

Mencetak setiap baris barang (`b.id` , `b.nama` , `b.harga` , `b.stok`) hasil pembacaan dari PostgreSQL.

03 **Tampilan awal bersih**

Saat pertama kali dibuka sebelum ada barang yang disimpan, tabel tampil kosong dan siap diisi lewat formulir.

PERSIAPAN LAB 4 • HALAMAN FORMULIR INPUT

Struktur berkas `views/tambah-barang.ejs`

Menyediakan tiga kolom input yang dikirimkan ke rute `POST /barang/tambah`:

BERKAS `VIEWS/TAMBAH-BARANG.EJS`

```
<body class="container mt-5">
  <h2>Tambah Barang</h2>
  <form action="/barang/tambah" method="POST">
    <input type="text" name="nama"
           class="form-control mb-2"
           placeholder="Nama Barang">
    <input type="number" name="harga"
           class="form-control mb-2"
           placeholder="Harga">
    <input type="number" name="stok"
           class="form-control mb-2"
           placeholder="Stok">
    <button type="submit" class="btn btn-success">
      Simpan
    </button>
  </form>
</body>
```

01 `action="/barang/tambah"`

Mengirimkan data formulir dengan metode `POST` ke rute penyimpan barang di dalam `app.js`.

02 **Tipe input `text` dan `number`**

Atribut `type="number"` memastikan pengguna hanya mengetikkan angka pada kolom harga dan stok.

03 **Kesesuaian atribut `name`**

Ketiga nama input (`nama`, `harga`, `stok`) diambil langsung melalui `const { nama, harga, stok } = req.body`.

PRAKTIK MANDIRI • LAB 4 HARI 2

Saatnya praktik: manajemen barang dengan PostgreSQL Cloud

Ikuti langkah **Lab 4: CRUD Operation** pada proyek Express Anda:

1. Buat berkas `views/barang.ejs` untuk menampilkan tabel daftar barang dan tombol **Tambah Barang**.
2. Buat berkas `views/tambah-barang.ejs` berisi formulir input `nama`, `harga`, dan `stok`.
3. Buka `app.js`, impor `PrismaClient`, lalu tambahkan rute `GET /barang`, `GET /barang/tambah`, dan `POST /barang/tambah`.
4. *Restart* server, lalu buka dan uji aplikasi Anda di alamat:

```
http://localhost:3000/barang
```

Cobalah menambahkan beberapa barang, lalu *restart* server Node.js Anda untuk membuktikan datanya tetap tersimpan permanen di *database cloud*!

TARGET LAB 4

Data tersimpan permanen di cloud

Setiap barang yang diinput lewat `/barang/tambah` langsung muncul pada tabel `/barang` dan tersimpan di PostgreSQL Render.

SESI BERIKUTNYA

Pada **Materi 5 Hari 2** (sesi penutup Hari 2), kita akan merapikan tampilan dengan mempelajari **Code Reusability** menggunakan komponen *Partials* di EJS.

HARI 2 • MATERI 5

Code reusability dan komponen Partials di EJS

Menerapkan prinsip DRY (*Don't Repeat Yourself*) dengan memecah bagian *header*, *navbar*, *footer*, dan kotak notifikasi menjadi komponen modular yang mudah dipelihara.

Kode: TX-BE01 **40 menit materi • 50 menit Lab 5**

POKOK BAHASAN SESI INI

01 Masalah pengulangan kode HTML & prinsip DRY

02 Konsep *Partials* (`header` & `footer`) di EJS

03 Perbedaan tag `<%-%>` dan `<%=%>` pada `include()`

04 Komponen *Alert* sukses & gagal yang dapat dipakai ulang

05 Praktik Lab 5 & rangkuman capaian Hari 2

PETA BELAJAR

Lima bahasan utama penutup Hari 2

Merapikan struktur antarmuka aplikasi agar tetap bersih dan mudah dirawat saat jumlah halaman bertambah.

01 Prinsip DRY

Menghindari penyalinan berulang (*copy-paste*) bagian `<head>`, `<nav>`, dan `<footer>` di setiap halaman.

02 Partials (`header` & `footer`)

Menyimpan potongan tata letak umum di dalam folder khusus `views/partials/`.

03 Sintaks `<%- include(...) %>`

Memahami alasan pemanggilan *partial* menggunakan tanda hubung (`-`), bukan tanda sama dengan (`=`).

04 Komponen *Alert* notifikasi

Membuat *partial* kotak pesan (`alert-success` / `alert-danger`) sebagai konfirmasi visual setelah aksi CRUD.

05 Praktik Lab 5 & rangkuman Hari 2

Memecah kode halaman `views/barang.ejs` menjadi modular lengkap dengan Navbar dan Footer standar.

01 • MENGGUNAKAN PARTIALS (HEADER/FOOTER)

Mengapa menyalin <head> dan <nav> berulang kali itu merepotkan?

Saat aplikasi berkembang memiliki halaman Beranda, Karyawan, Barang, dan Tambah Barang, menyalin rangka HTML yang sama menyulitkan pemeliharaan kode:

TANPA PARTIALS • COPY-PASTE DI TIAP BERKAS

Sulit diperbarui saat ada perubahan

Jika aplikasi memiliki 10 halaman EJS dan kita ingin menambah satu menu baru di Navbar, kita terpaksa membuka dan mengedit ke-10 berkas tersebut satu per satu.

DENGAN PARTIALS • PRINSIP DRY (DON'T REPEAT YOURSELF)

Ubah di satu tempat, semua halaman ikut berubah

Bagian `<head>`, Navbar, dan `<footer>` disimpan satu kali di dalam folder `views/partials/` lalu dipanggil di setiap halaman dengan `include`.

> Prinsip **DRY (*Don't Repeat Yourself*)** memastikan setiap potongan tata letak umum hanya ditulis pada satu berkas sumber.

01 • ARSITEKTUR LAYOUT MODULAR

Memecah satu halaman utuh menjadi tiga bagian terpisah

Dengan fitur **Partials** di EJS, setiap berkas halaman utama cukup menuliskan konten intinya saja:

01 BAGIAN ATAS

`partials/header.ejs`

Menyimpan pembuka `<!DOCTYPE html>`, `<head>`, tautan CSS Bootstrap, pembuka `<body>`, dan bilah `<nav>`.

Dipanggil di baris paling atas setiap halaman.

02 KONTEN UTAMA

`views/barang.ejs`

Hanya berisi elemen unik milik halaman tersebut, seperti judul halaman, tombol tambah, dan tabel daftar barang.

Kode halaman menjadi jauh lebih ringkas dan bersih.

03 BAGIAN BAWAH

`partials/footer.ejs`

Menyimpan elemen `<footer>` hak cipta, skrip JS Bootstrap, serta tag penutup `</body>` dan `</html>`.

Dipanggil di baris paling bawah setiap halaman.

01 • SINTAKS PEMANGGILAN PARTIAL

Perbedaan tag <%- %> dan <%= %> pada EJS

Saat menyisipkan berkas *partial*, perhatikan bahwa kita menggunakan tanda hubung (`<%-`) dan bukan tanda sama dengan (`<%=`):

SINTAKS TAG EJS	CARA KERJA PADA KODE HTML	KAPAN DIGUNAKAN?
`<%= variabel %>`	Mengubah tanda kurung sudut HTML (`<` , `>`) menjadi teks biasa demi keamanan.	Mencetak nilai variabel teks atau angka dari server (misalnya `<%= b.nama %>`).
`<%- include('partials/header') %>`	Merender isi berkas sebagai elemen HTML utuh tanpa diubah menjadi teks biasa.	**Wajib digunakan saat memanggil komponen *Partials*** (`header` , `footer` , `alert`).
`<% logika %>`	Menjalankan baris logika JavaScript tanpa mencetak teks ke halaman.	Menuliskan pengulangan `forEach` atau kondisi `if / else` .

> **Tips penting:** Jika Anda keliru menulis `<%= include(...) %>` dengan tanda =, peramban akan menampilkan kode HTML mentah sebagai teks di layar.

01 • ISI BERKAS PARTIALS

Membedah pasangan header.ejs dan footer.ejs

Tag pembuka `<html>` dan `<body>` diletakkan di `header.ejs`, sedangkan tag penutupnya diletakkan di `footer.ejs`:

BERKAS `VIEWS/PARTIALS/HEADER.EJS`

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Aplikasi Node Modern</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
  <div class="container">
    <a class="navbar-brand" href="/">MyApp</a>
    <div class="collapse navbar-collapse">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item">
          <a class="nav-link" href="/barang">Barang</a>
        </li>
      </ul>
    </div>
  </div>
</nav>
```

BERKAS `VIEWS/PARTIALS/FOOTER.EJS`

```
<footer class="text-center mt-5 p-3 bg-light">
  <p class="mb-0">
    &copy; 2024 Aplikasi Modern. Hak cipta dilindungi.
  </p>
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>
```

Ketika keduanya mengapit konten halaman utama, terbentuklah satu dokumen HTML yang utuh.

02 • KOMPONEN ALERT SUKSES & GAGAL

Membuat *partial* kotak notifikasi `alert-success` dan `alert-danger`

Untuk memberikan pengalaman pengguna (UX) yang baik setelah operasi CRUD, kita dapat membuat satu *partial* khusus berisi kotak pemberitahuan:

01 **Konfirmasi visual yang jelas**

Memberi tahu pengguna secara langsung saat barang baru berhasil disimpan (hijau) atau saat terjadi kesalahan input (merah).

02 **Cukup dibuat satu kali**

Disimpan sebagai `views/partials/alert.ejs` sehingga tidak perlu menulis ulang kode `div.alert` di setiap halaman.

03 **Tampil hanya jika ada status**

Dilengkapi pengecekan kondisi `if` agar kotak peringatan hanya muncul ketika server mengirimkan pesan status.

CONTOH `VIEWS/PARTIALS/ALERT.EJS`

```
<% if (typeof sukses !== 'undefined' && sukses) {
%>
  <div class="alert alert-success">
    <%= sukses %>
  </div>
<% } %>

<% if (typeof gagal !== 'undefined' && gagal) {
%>
  <div class="alert alert-danger">
    <%= gagal %>
  </div>
<% } %>
```

Dipanggil di halaman mana saja dengan `<%- include('partials/alert') %>`.

02 • INTEGRASI ALERT DENGAN RUTE CRUD

Menampilkan pesan konfirmasi setelah `res.redirect()`

Saat penyimpanan barang di rute `POST` selesai, kita dapat menitipkan pesan singkat ke halaman tabel melalui parameter URL:

PENGIRIMAN STATUS DI `APP.JS`

```
// 1. Setelah berhasil menyimpan barang baru:
res.redirect('/barang?pesan=Barang+berhasil+ditambahkan');

// 2. Pada rute GET /barang:
app.get('/barang', async (req, res) => {
  const barangs = await prisma.barang.findMany();
  res.render('barang', {
    listBarang: barangs,
    sukses: req.query.pesan
  });
});
```

01 **Titipkan pesan di `redirect`**

Alamat `/barang?pesan=Barang+berhasil+ditambahkan` membawa informasi status menuju rute `GET /barang`.

02 **Baca melalui `req.query.pesan`**

Jika pengguna membuka `/barang` secara normal tanpa parameter, nilai `req.query.pesan` otomatis kosong (`undefined`).

03 **Direnderi oleh `partials/alert`**

Ketika `sukses` berisi teks pesan, kotak hijau `alert-success` langsung tampil di bagian atas halaman.

SIMULASI INTERAKTIF • PERAKITAN LAYOUT EJS

Perbandingan kode halaman sebelum dan sesudah memakai *Partials*

1. Sebelum Refactoring | 2. Sesudah Memakai Partials | 3. Struktur Folder views/

KONDISI AWAL • SEBELUM LAB 5

Setiap berkas EJS menulis ulang tag head, link Bootstrap, dan body

Pada Lab 4, baik barang.ejs maupun tambah-barang.ejs masing-masing menuliskan tag <!DOCTYPE html>, <head>, dan <link> secara berulang.

KODE LAMA VIEWS/BARANG.EJS

```
<!DOCTYPE html>
<html lang="id">
<head>
  <link href="https://cdn.jsdelivr.net/.../bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
  <h2>Manajemen Barang</h2>
  <!-- Tabel barang di sini -->
</body>
</html>
```

Belum memiliki Navbar dan Footer yang seragam antarhalaman.

SIMULASI INTERAKTIF • PERAKITAN LAYOUT EJS

Perbandingan kode halaman sebelum dan sesudah memakai *Partials*

1. Sebelum Refactoring | **2. Sesudah Memakai Partials** | 3. Struktur Folder views/

KONDISI BARU • DENGAN PARTIALS

Kode views/barang.ejs menjadi jauh lebih bersih dan fokus pada isi

Bagian atas dan bawah digantikan oleh pemanggilan <%- include('partials/header') %> dan <%- include('partials/footer') %>.

KODE BARU VIEWS/BARANG.EJS (LAB 5)

```
<%- include('partials/header') %>

<div class="container">
  <h2>Manajemen Barang</h2>
  <a href="/barang/tambah" class="btn btn-primary mb-3">Tambah
Barang</a>
  <!-- Tabel barang di sini -->
</div>

<%- include('partials/footer') %>
```

Otomatis menampilkan Navbar gelap di atas dan Footer di bawah halaman.

SIMULASI INTERAKTIF • PERAKITAN LAYOUT EJS

Perbandingan kode halaman sebelum dan sesudah memakai *Partials*

1. Sebelum Refactoring | 2. Sesudah Memakai Partials | **3. Struktur Folder views/**

ORGANISASI BERKAS • FOLDER VIEWS/

Susunan folder views/ yang rapi setelah menerapkan Partials

Seluruh potongan komponen umum berkumpul di dalam subfolder views/partials/, terpisah dari halaman-halaman utama.

STRUKTUR FOLDER VIEWS/

```
views/
├── partials/
│   ├── header.ejs        <-- Head + Navbar Bootstrap
│   └── footer.ejs        <-- Footer + Script JS Bootstrap
├── barang.ejs            <-- Halaman daftar barang
├── tambah-barang.ejs     <-- Halaman form tambah barang
├── karyawan.ejs          <-- Halaman dari Hari 1
└── index.ejs             <-- Halaman beranda
```

Semua halaman kini dapat memakai `header.ejs` dan `footer.ejs` yang sama.

PERSIAPAN LAB 5 • REFACTORING KODE VIEW

Mengubah `views/barang.ejs` agar menggunakan *Partials*

Hapus tag `<html>`, `<head>`, dan `<body>` lama pada `views/barang.ejs`, lalu apit kontennya dengan `include`:

BERKAS `VIEWS/BARANG.EJS` (VERSI LAB 5)

```
<%- include('partials/header') %>

<div class="container">
  <h2>Manajemen Barang</h2>
  <a href="/barang/tambah" class="btn btn-primary mb-3">
    Tambah Barang
  </a>
  <table class="table table-bordered">
    <thead>
      <tr><th>ID</th><th>Nama</th><th>Harga</th><th>Stok</th>
</tr>
    </thead>
    <tbody>
      <% listBarang.forEach(b => { %>
        <tr>
          <td><%= b.id %></td>
          <td><%= b.nama %></td>
          <td><%= b.harga %></td>
          <td><%= b.stok %></td>
        </tr>
      <% }) %>
    </tbody>
  </table>
</div>

<%- include('partials/footer') %>
```

01 **Baris pembuka** header

`<%- include('partials/header') %>` otomatis menyisipkan `<head>` dan Navbar gelap di bagian atas halaman.

02 **Pembungkus** `<div class="container">`

Menjaga agar judul dan tabel barang memiliki batas tepi kiri-kanan yang sejajar dengan isi Navbar.

03 **Baris penutup** footer

`<%- include('partials/footer') %>` menyisipkan kaki halaman hak cipta dan menutup tag dokumen HTML.

PRAKTIK MANDIRI • LAB 5 HARI 2

Saatnya praktik: memecah *layout* dengan komponen *Partials*

Ikuti langkah **Lab 5: Komponen Partials** pada proyek Express Anda:

1. Di dalam folder `views/`, buat folder baru bernama `partials`.
2. Buat berkas `views/partials/header.ejs` berisi tag `<head>` dan Navbar navigasi Bootstrap.
3. Buat berkas `views/partials/footer.ejs` berisi elemen `<footer>`, skrip JS Bootstrap, dan penutup `</body></html>`.
4. Perbarui `views/barang.ejs` agar menggunakan `<%- include('partials/header') %>` dan `<%- include('partials/footer') %>`.
5. Segarkan peramban Anda pada alamat berikut:

```
http://localhost:3000/barang
```

Tips tambahan: Terapkan juga pemanggilan `header` dan `footer` yang sama pada berkas `views/tambah-barang.ejs`!

TARGET LAB 5

Tampilan rapi dengan Navbar & Footer

Halaman `/barang` kini memiliki bilah navigasi gelap bertuliskan **MyApp** di bagian atas serta kaki halaman hak cipta di bagian bawah.

TANTANGAN MANDIRI

Tambahkan tautan menu `<a class="nav-link" href="/karyawan">Karyawan</a>` di `partials/header.ejs` dan lihat perubahannya di semua halaman!

PENUTUP HARI 2 • RANGKUMAN & LANGKAH BERIKUTNYA

Capaian belajar Hari 2 dan agenda Hari 3

Selamat! Hari ini aplikasi kita telah terhubung penuh dengan *database cloud* dan memiliki kode antarmuka yang modular:

MATERI 1

Form Input & Validasi

Membaca data formulir `POST` dengan `express.urlencoded` dan memeriksa keamanan input menggunakan `express-validator`.

MATERI 2

PostgreSQL Cloud & .env

Menyiapkan *instance* PostgreSQL gratis di Render.com dan mengamankan *External Database URL* di dalam berkas `.env`.

MATERI 3

Prisma ORM & Migration

Mendefinisikan model `Barang` di `schema.prisma` dan membentuk tabel fisik di Render lewat `npx prisma migrate dev`.

MATERI 4 & 5

Fitur CRUD & Partials EJS

Mengelola data barang dengan `PrismaClient` serta memecah tata letak Navbar dan Footer menggunakan komponen *Partials*.

Agenda Hari 3 Besok: Membangun sistem **Authentication & Authorization (Login)**, **Relasi Antar-Tabel (Multi-table)**, fitur **Searching & Filtering**, serta **REST API Dasar**!